

《云计算与大数据处理》

实验报告

实验名称	云端数据接口与可复现环境
实验日期	2026 年 10 月 25 日
专业	
班级	
学号	2410311319
姓名	彭博
指导教师	

实验一 云端数据接口与可复现环境实验报告

一 实验目的与完成情况

实验一要把本地生成的事件文件上传到对象存储，再下载回来，检查文件有没有变化。课程使用 Moto 在本机提供 S3 接口，所以这里操作的是一个临时的模拟服务。我还要分别改对象键和文件内容，观察两种改动对 SHA-256 的影响。

最后我跑通了上传、两次写入和下载。原始文件与下载文件的 SHA-256 相同，程序输出 hash_matches=true；两次写入的版本 ID 不同，输出 version_ids_differ=true。这些结果说明本次本机上传下载的文件字节一致，不能据此说真实云存储也经过了测试。

二 实验环境与数据准备

我按课程的 requirements.lock.txt 安装依赖，并在项目虚拟环境里运行。pip check 输出 No broken requirements found.。我记录下实际版本，方便以后重新跑：

项目	本次记录
系统	Windows NT 10.0，构建号 26300，64 位
Python	3.12.14；项目解释器为 .venv\Scripts\python.exe
主要依赖	boto3 1.43.93，Moto 5.2.3
随机种子	42
原始文件	data/raw/events.jsonl；20,102 行，3,892,184 字节

生成数据时指定 20,000 条有效且不重复的事件。生成器还加入了 100 条重复事件和 2 条异常事件，所以文件一共是 20,102 行。这个文件是 JSONL 格式，每一行都是一个 JSON 对象。我查看首行，里面有 event_id、event_time、service、model、input_tokens、output_tokens、latency_ms、status 等字段；首行的 event_id=evt-00000000、service=assistant、input_tokens=1538、status=error。我没有删除重复和异常行，实验二还要用这份原始文件做质量检查。

三 实验原理与预期结果

这里有三个容易弄混的东西：data/raw/events.jsonl 是电脑上的文件路径；bdp-teaching 是桶名；bronze/events/<内容 SHA-256>.jsonl 是对象键。对象键里虽然有斜杠，但它不是电脑上的文件夹。SHA-256 则是根据文件字节计算的摘要。

改个人脚本前，我先写下预测，再拿运行结果对照：

我改了什么	原先预计	跑完以后看到
只把对象键从共享键换成学号前缀	文件没变，SHA-256 应该不变	两次都是 23c0b20c…ba49f0e8
把 payload 的 tokens 从 120 改成 121	文件字节变了，SHA-256 应该改变	变成 1d0f0d28…0ee63bec
上传后再下载	如果收到的字节没变，两边摘要应相同	三次对照都是 equal=true

这里核对的是字节，不是事件内容是否合理。哪怕只是改变 JSON 的缩进，也可能改变字节和 SHA-256；反过来，摘要相同也不能证明负 Token 数这样的业务值合理。

四 实验过程与运行结果

我先启动本机 Moto HTTP 服务并创建 bdp-teaching 桶，然后把事件文件上传到对象键，向同一个键再写入一次，最后下载到 outputs/lab01/downloaded-events.jsonl。最后一次完整运行的 run_id 是 51a0dcfa88ed47f1abe9a4d94e9724bf。

项目	实际结果
本次 endpoint	http://127.0.0.1:51091，只在这次临时服务运行时有效
对象键	bronze/events/0d93bcd25e28047f804238c220bdb5b1d2dadf328647c07de85ba0104e8e2854.jsonl
原始和下载文件的 SHA-256	都是 0d93bcd25e28047f804238c220bdb5b1d2dadf328647c07de85ba0104e8e2854
程序核对	hash_matches=true，version_ids_differ=true

上传接口返回成功只能说明请求被处理。下载后再算一次 SHA-256，才能核对这次取回的字节是否和本地文件一样。两次写入的版本 ID 不同，说的是对象版本；文件内容的摘要没变，说的是内容，两者并不冲突。

五 参数修改与结果对比

我先原样运行课程附件，得到共享对象键 shared/raw/demo.json，内容摘要是 23c0b20c65b4476c7e541044c09c88cd970bb02191681f804daef0b7ba49f0e8，run_id=0269b2bdbc4b4b4b931362062e03c086。

第一次只改 make_key()，让它返回 students/{STUDENT_ID}/raw/demo.json。脚本从课程身份登记读取学号，代码里没有写死我的学号。实测对象键变成了 students/2410311319/raw/demo.json，摘要仍是

23c0b20c65b4476c7e541044c09c88cd970bb02191681f804daef0b7ba49f0e8，run_id=a507107c4a3b4b71bd836f62d987e8ee。我由此确认：换存放位置不会改变文件本身的哈希。加上学号前缀也能避免大家都写到同一个共享键。

第二次保留新对象键，把 payload 的 tokens 从 120 改为 121。新摘要变为 1d0f0d28612d93ec866d670eab8876e9dfb64d1c2b2cb84ee24fa60d0ee63bec，与改动前不同；但这次上传与下载仍相等，run_id=eab0ae15a2a0418bb795f379bd1a3da6。两次改动可以在 object_key_change.diff 和 final_changes.diff 里看到。

六 故障模拟与恢复

为了看连接失败时会发生什么，我把 endpoint 指到 http://127.0.0.1:1。程序报 EndpointConnectionError: Could not connect to the endpoint URL，外层 capture 退出码是 1，run_id=4d334a30e08d46d88a934a4a2226aae4。这个地址没有实验所需的 S3 服务，错误发生在连接阶段，不能把它当作数据格式或权限检查的结果。

改回默认的临时 Moto 服务后，capture 退出码恢复为 0，run_id=db7c975260bc4f59b020822409bb8159。我又独立跑了一次主程序，得到第四节的结果。原来的报错记录保留在证据文件夹里，方便和恢复后的结果对照。

七 实验结论与局限

这次能确认三件事：本机 Moto 的 S3 接口接收了建桶、写入和读取请求；原始文件和下载文件在这次运行中有相同的 SHA-256；对同一键写两次拿到了不同的版本 ID。

还有一些事这次没有测试。数据里本来就有 2 条异常事件，能上传不等于字段都合法。Moto 是本机临时服务，不能用它证明真实云服务的持久性、跨机器容灾、IAM 权限或费用。这里用的 testing 也是本机模拟凭据。实验二会继续读本地保留的 data/raw/events.jsonl，处理重复和异常记录。如果以后接真实云服务，这些权限、存储和费用问题都要重新检查。

八 实验材料与代码仓库

提交包的 evidence/ 里保留了 11 组原生 JSON、同名 HTML 和 Chrome 截图。附录中的五张图直接取自这些文件，没有拼接或改字。artifacts/ 放了原始与下载 JSONL、最终 storage.json 等文件，运行前的判断另存于 prediction.md。这些材料可以按运行编号对照本报告，但截图和实验结果还需要老师复核。

代码仓库：https://github.com/Pb-Carl/- 。这次配置环境、整理命令、改脚本和写报告时用到了 AI 助手；行数、哈希、错误类型和运行编号来自本机文件。提交前还需要我核对代码和截图，老师复核后才算完成验收。

附录 运行结果截图

以下图片保留原始证据页的学号、实验编号、运行编号和结果。放大查看或逐项核对时，可以打开提交包 evidence/ 中的同名 PNG、HTML 和 JSON。

图 1 主程序：原始与下载 SHA-256 相同，版本 ID 不同

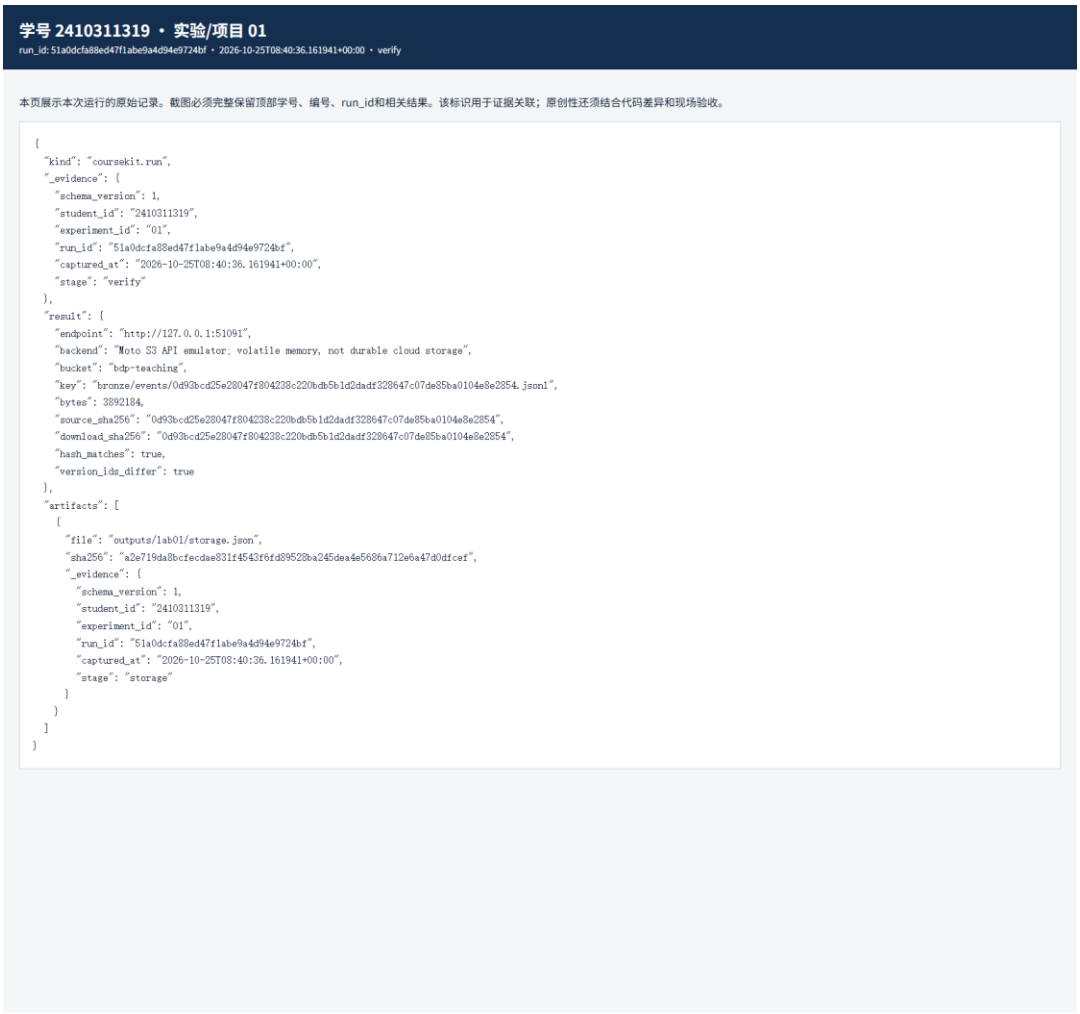


图 2 只换对象键：内容摘要保持不变



图 3 tokens 改为 121：摘要改变，上传下载仍相等



图 4 错误地址：连接失败，退出码为 1

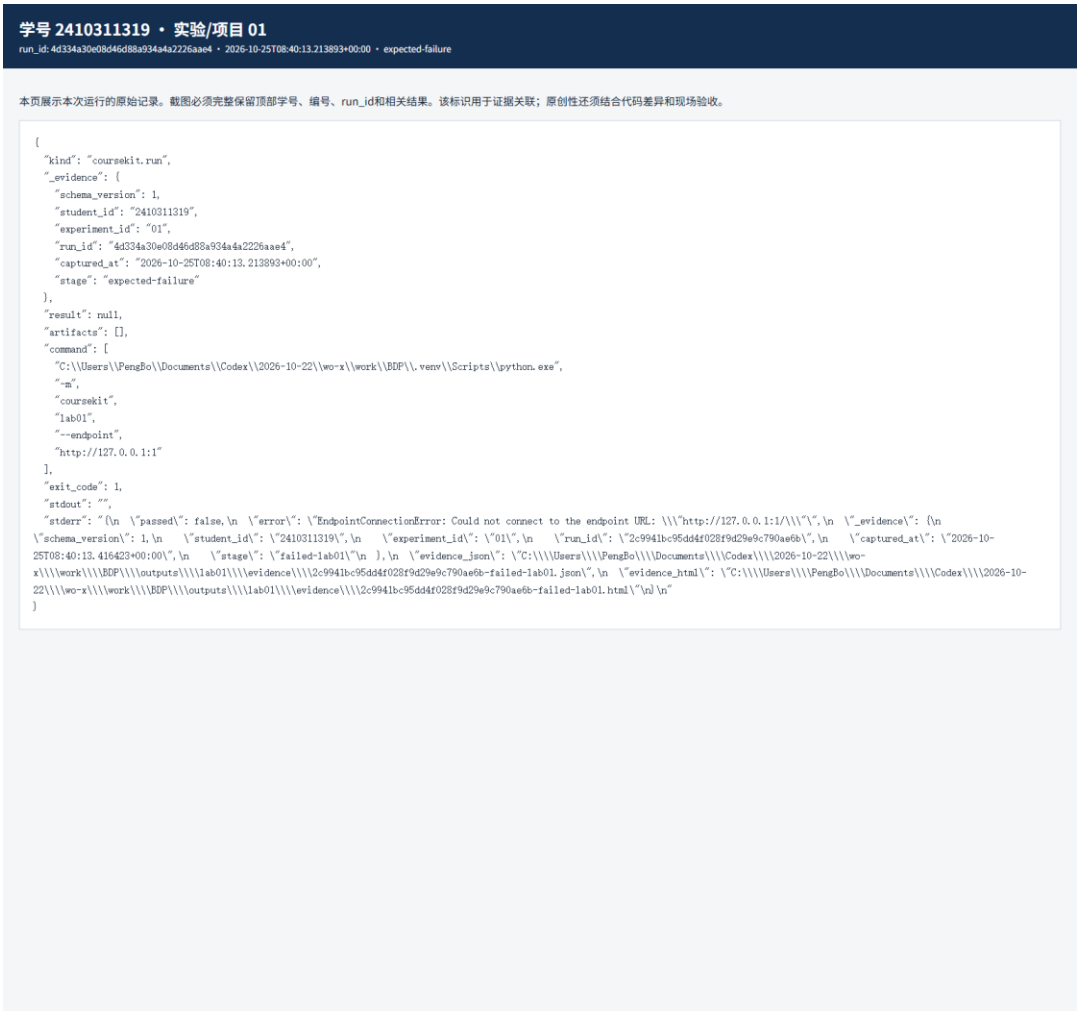


图 5 改回服务地址：退出码恢复为 0

